

Project Playbook

Integrated Secure Data Erasure & Advanced File Recovery Tool for Digital Forensics and Data Sanitization

1. Project Overview

Project Name: Purgent

Problem Statement: Design and Development of an Integrated Secure Data Erasure and Advanced File Recovery Tool for Digital Forensics and Data Sanitization.

Core Idea: Integrate two seemingly opposite capabilities into a single platform:

- **Secure Data Sanitization** — permanently and unrecoverably destroy data.
- **Forensic-Grade Recovery** — recover evidence from deleted, corrupted, or formatted media.

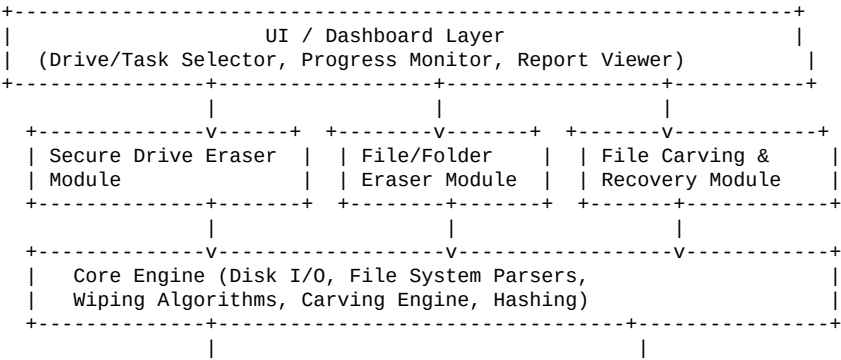
Today these two capabilities live in separate tools (e.g., DBAN/Eraser for wiping, Autopsy/FTK/EnCase for recovery). The goal of this project is to build a unified, cross-platform, standards-compliant tool that serves investigators, enterprises, government agencies, and individuals alike.

2. Goals & Success Criteria

Goal	Success Metric
Secure erasure of drives	NIST 800-88 / DoD 5220.22-M compliant wipe, verified unrecoverable via forensic rescan
File/folder level secure delete	Metadata + residual traces removed, verified with hex/forensic check
File carving & recovery	High recovery rate on formatted/corrupted media, fragmented file reconstruction working
Reporting & audit	Tamper-evident, exportable (PDF/JSON/XML) reports with hash-based integrity
Usability	Non-forensic-expert can operate via GUI in < 5 minutes learning curve
Compliance	Mapped explicitly to at least 2 recognized standards (see Section 6)

3. System Architecture

3.1 High-Level Modules



+-----v-----+	+-----v-----+
Audit Logging & Reporting	Storage/Device Abstraction
(tamper-evident, signed)	(HDD/SSD/USB/SD/NVMe layer)
+-----+	+-----+

3.2 Suggested Tech Stack

- **Core/backend engine:** C++ or Rust (low-level disk access, performance, memory safety for forensic integrity — Rust is preferred for safety-critical carving code)
- **Cross-platform layer:** Python (orchestration, scripting, plugin modules) or Go
- **GUI:** Electron / Qt (C++) / Avalonia (.NET) — pick one that supports Windows + Linux (minimum); macOS optional
- **Low-level disk access:** libparted, hdparm/nvme-cli wrappers (Linux), Win32 DeviceIoControl APIs (Windows)
- **File system parsers:** Custom parsers or reuse open-source forensic libraries (TSK/Sleuth Kit as reference, not copy — build original implementation to avoid licensing conflicts)
- **Hashing/Integrity:** SHA-256/SHA-3 for report signing, HMAC for tamper-evidence
- **Reporting:** PDF generation (e.g., reportlab equivalent in chosen stack), JSON/XML structured logs
- **Database (case management):** SQLite (local) for audit trail storage

4. Module-Wise Breakdown

4.1 Module 1 — Secure Drive Eraser

Scope: HDD, SSD, USB, memory cards, external drives.

Key Features:

- Multiple wiping standards: NIST 800-88 (Clear/Purge), DoD 5220.22-M (3-pass/7-pass), Gutmann (35-pass, optional/legacy), single-pass zero/random for SSD (TRIM-aware)
- SSD-specific handling: ATA Secure Erase, NVMe Sanitize/Format commands (since overwrite-based wiping is unreliable on SSD due to wear-leveling)
- HPA/DCO detection and removal (Host Protected Area / Device Configuration Overlay) — data often hides here
- Bad sector handling (skip + log, don't fail whole operation)
- Pre-wipe device detection: model, serial, capacity, media type auto-detect
- Post-wipe verification: random sampling read-back to confirm no recoverable data
- Certificate of erasure generation (digitally signed PDF + JSON)

Standards to Map:

- NIST SP 800-88 Rev.1 (Guidelines for Media Sanitization)
- DoD 5220.22-M
- IEEE 2883-2022 (newer sanitization standard, mention if time permits)

4.2 Module 2 — Secure File & Folder Eraser

Scope: Selective, targeted secure deletion within a live file system.

Key Features:

- Multi-pass overwrite of file content before unlink (respecting SSD constraints — flag to user that SSD file-level wipe is less reliable than full-drive wipe)
- Metadata scrubbing: filenames in MFT/journal entries, thumbnails, recent-file lists, registry traces (Windows), .bash_history/.local/share/Trash (Linux), Spotlight/metadata (macOS)
- Batch/recursive folder operations with progress tracking
- Cross-filesystem support: NTFS, exFAT, FAT32, ext4, APFS (read/analysis at least, delete on native OS)
- Slack space and residual cluster clearing for the erased file's previous location
- Verification: attempt recovery scan post-deletion to confirm irrecoverability
- Per-operation audit log entry

4.3 Module 3 — Advanced File Carving & Recovery

Scope: Recovery from formatted, corrupted, or damaged media — without relying on file system metadata.

Key Features:

- Signature-based carving: file header/footer matching (JPEG, PNG, PDF, DOCX/ZIP-based formats, MP4, etc.)
- Structure-based carving: understanding internal file structure (e.g., JPEG segment markers, PDF xref tables) to validate carve boundaries beyond simple header/footer guess

- Fragmented file reconstruction: heuristics for handling non-contiguous fragments (e.g., bifragment gap carving)
- Metadata-less recovery: raw sector/cluster scanning when MFT/inode tables are destroyed
- Automatic classification: ML-based or rule-based classifier to tag recovered files by type/content category
- Confidence scoring: each recovered file gets a score (e.g., header valid + footer valid + internal structure valid = high confidence; header-only = low confidence)
- Evidential integrity: write-blocking on source media (read-only mount), hash every recovered file (SHA-256) immediately, chain-of-custody log

5. Reporting & Audit Management System

- Every operation (erase or recover) generates a signed, timestamped, tamper-evident report
- Report contents: operation type, device/file details, standard used, start/end time, operator ID, verification result, hash of report itself
- Export formats: PDF (human-readable), JSON/XML (machine-readable, for integration with case management systems)
- Centralized audit log viewable from dashboard, filterable by date/operation/device
- Optional: blockchain-style hash-chaining of logs for extra tamper-evidence (nice-to-have, not core MVP)

6. Standards & Compliance Mapping

Standard	Applies To	Notes
NIST SP 800-88 Rev.1	Drive erasure	Clear / Purge / Destroy categories
DoD 5220.22-M	Drive/file erasure	Legacy but widely referenced
IEEE 2883-2022	Drive erasure	Modern replacement addressing SSD/flash
ISO/IEC 27001	Overall data handling/audit	Organizational compliance angle
Digital Forensics: ACPO Principles / ISO/IEC 27037	Recovery module	Evidence handling, chain of custody

For judges/mentors, build an explicit "Compliance Matrix" slide/table showing which module follows which standard.

7. Development Roadmap (Hackathon-Style Phased Plan)

Phase 0 — Research & Design (Week 1)

- Study NIST 800-88, DoD 5220.22-M specs in detail
- Study existing tools (DBAN, Eraser, Autopsy, PhotoRec/TestDisk, Sleuth Kit) — as reference/benchmark, not for copying code
- Finalize tech stack, architecture diagram, DB schema for audit logs
- Define file signature database (magic numbers) for carving module

Phase 1 — Core Engine + Drive Eraser MVP (Week 2-3)

- Disk detection & abstraction layer (cross-platform)
- Implement single-pass and 3-pass overwrite algorithms
- Implement ATA Secure Erase call for SSD (or simulate if hardware unavailable in demo)
- Basic verification (read-back sampling)
- CLI-level working prototype

Phase 2 — File/Folder Eraser Module (Week 3-4)

- File-level secure delete implementation
- Metadata scrubbing logic
- Batch operation support
- Integration with audit logging

Phase 3 — File Carving & Recovery Module (Week 4-6)

- Signature-based carving for 5-10 common file types (start small: JPEG, PNG, PDF, DOCX, MP4)
- Structure validation logic
- Basic fragmented-file heuristic (even a simple gap-carving demo is impressive)
- Confidence scoring logic
- Classification (rule-based first; ML classifier as stretch goal)

Phase 4 — Reporting, Audit & GUI Integration (Week 6-7)

- Unified dashboard connecting all 3 modules
- Report generation (PDF/JSON)
- Signed/tamper-evident logging
- Polish UI/UX (progress bars, device selection wizard, drag-drop for file erase)

Phase 5 — Testing, Validation & Documentation (Week 7-8)

- Test on sample images/virtual disks (use test VMs, not real production drives!)
- Validate wipe effectiveness using forensic tools (e.g., attempt recovery post-wipe using your own carving module — good demo point)
- Validate recovery module against known-good test datasets (e.g., NIST CFReDS forensic test images — publicly available for exactly this purpose)

- Write user manual, technical documentation, performance evaluation report

8. Team Roles (Suggested Split for Hackathon Team of 5-6)

Role	Responsibility
Team Lead / Architect	Overall design, integration, standards research
Systems/Low-level Dev	Drive eraser module, disk I/O, OS-level APIs
Forensics/Carving Dev	File carving, signature DB, fragment reconstruction
Backend/Data Dev	Audit logging, reporting, database, hashing/integrity
Frontend/UX Dev	Dashboard, GUI, visualization of progress/reports
QA/Documentation	Testing, validation datasets, user manual, PPT/pitch

9. Demo Strategy (For Judges)

- **Live wipe demo:** Wipe a small USB drive with NIST 800-88 method → show verification pass → show generated erasure certificate.
- **Live recovery demo:** Take a pre-formatted test image (prepared beforehand with known deleted files) → run carving module → show recovered files with confidence scores → show forensic report.
- **Unified dashboard walkthrough:** Show both operations logged in one audit trail — this is your key differentiator vs. existing single-purpose tools.
- **Compliance slide:** Quickly show the standards mapping table.

10. Risks & Mitigations

Risk	Mitigation
SSD wipe unreliability (wear-leveling)	Clearly document limitation, use ATA Secure Erase/NVMe Sanitize instead of blind overwrite
Real hardware access limited in dev environment	Use virtual disks / disk image files (.img/.vhd) for development & demo
File carving accuracy on complex fragmented files	Start with simple file types, expand gradually; be honest about confidence scoring rather than overclaiming 100% recovery
Cross-platform low-level API differences	Abstract early via a hardware-access interface layer; prioritize one OS (e.g., Windows or Linux) for MVP, add others after
Legal/ethical use of forensic tool	Add clear licensing/usage disclaimer; log operator identity for accountability
Time constraints (hackathon)	Ruthlessly scope MVP — 5-10 file types for carving, 2 standards for wiping, is enough to demo the integration concept

11. Deliverables Checklist

- Secure Drive Eraser Module (working prototype)
- Secure File & Folder Eraser Module (working prototype)
- Advanced File Carving & Recovery Module (working prototype)
- Unified Reporting & Audit Management System
- GUI Dashboard integrating all modules
- Validation & testing documentation (with test datasets used)
- User manual
- Technical documentation (architecture, APIs, algorithms used)
- Performance evaluation report (wipe speed, recovery rate %, false positive rate)
- Compliance mapping document
- Pitch deck / demo script

12. Key Differentiators to Emphasize in Pitch

- Unified platform — first tool combining sanitization + recovery in one audit trail (reduces tool-switching for investigators)
- Standards-first design — explicit NIST/DoD/IEEE compliance, not just "secure delete" marketing claims
- Confidence-scored recovery — transparency about recovery certainty instead of black-box results
- Tamper-evident reporting — court-admissible-style audit trail (signed hashes, timestamps)
- Cross-device, cross-filesystem support in a single tool

Playbook prepared as an execution guide — adapt module scope and timeline to your actual team size and hackathon duration.